

frontmatter — the plan

The file is the only source of truth. Every board, calendar, decision card, dashboard and AI edit is a deterministic, reversible projection of it — owning no state of its own.

You will say that we already have a master plan document, and that I wrote it yesterday, so why sit and rewrite the whole thing again. Fair question. The reason is that yesterday's document was a *findings* document — it was organised around the six rounds of research that produced it, which is the right shape for me and the completely wrong shape for anybody else who has to read it. Since then we also ran a sixth round that changed three things I was fairly confident about, including the price of the top tier and the one sentence that explains what this product even is. So this is not a polish pass. This is the plan rewritten around the *product*, with the research sitting underneath it instead of on top of it.

Everything here is grounded. Where a number came from running something on this machine I mark it `[measured]`. Where I opened the actual source I mark it `[fetched]`. Where it is a search summary that nobody has verified yet I mark it `[SS]` and you should treat it as a lead, not a fact. The full evidence — 35 agent reports across six rounds, roughly 6.2 million tokens of research — sits in `docs/research/agent-reports-2026-08-28/`. I am not going to repeat all of it here. I will point at it.

One more thing before we start. There are two settled decisions that I am not reopening in this document, because we already spent months on them and both survived every round since: **we are not inventing a new markdown format**, and **MDMAX is a library that sits under the editor, not a compiler programme with its own destiny**. Every extension we build is a profile over valid CommonMark that degrades to readable text in any dumb renderer. If you find yourself arguing with that, read `HANDOFF-mdz-markdown-format-2026-07-29.md` first.

Contents

01	The problem, stated properly
02	The one idea
03	What the market cannot do (and how long that lasts)
04	The trick: markdown doing app work without app compute
05	The product, layer by layer
06	Every screen
07	Who pays
08	Pricing
09	Marketing
10	The build order
11	What I will not build
12	What could kill this
13	What to read before writing code
14	Decisions that are yours
15	What is still unverified

§1 The problem, stated properly

Let me start with what is actually broken, because if I get this part wrong then everything downstream is decoration.

Your AI work evaporates. This is the big one and it is new. You spend an hour with Claude or ChatGPT working out a product decision, a plan, a piece of research. At the end of it there is a good answer sitting in a chat window. Then what. ChatGPT's only real escape hatch is a full-account ZIP that arrives by email with a link that dies in 24 hours [SS]. NotebookLM did not even *keep* your chats until January 2026 [SS]. There is an entire cottage industry of browser extensions built to scrape conversations out — one of them has over 400,000 installs [SS] — and every single one of them stops at the same place: a stale flat dump that is out of date the moment you ask a follow-up question. So the work does not disappear because nobody wanted to save it. It disappears because the save is a dead end.

Sync silently eats your files. This is the oldest pain in the category and it is still bleeding. In the July research it came out as the single most-mentioned complaint, 561 of 3,220 Hacker News comments [SS]. And these are not historical wounds — in the last six weeks alone the Obsidian forum has a thread about a file that showed "fully synced" while quietly missing its last ten Korean characters, and another about files disappearing on macOS [fetched]. Notice the shape of it. It is not "sync was down." It is "sync said green and lied."

Editors rewrite bytes you never touched. I used to think this was a library-choice problem — that people were just using a bad YAML serialiser. Then we actually executed three competitors' write paths and it turned out to be architectural. Front Matter CMS — the VS Code extension with 80,527 installs that owns our name [fetched] — deletes your YAML comments, resolves your anchors, and strips leading zeros when you edit a single title field. Its own source code has a comment saying "*Do our own parsing to keep the comments,*" and then it builds the comment-preserving document object and throws it away [measured]. Hubble.md deletes reference links along with their visible text and silently drops any frontmatter key with a colon in it, like `og:image` [measured]. OpenKnowledge, which is the most serious competitor on the board, permanently changes `tags: [alpha, beta]` into `tags: [ alpha, beta ]` on any frontmatter edit and never puts it back [measured, on their shipped build]. Three tools, three different code bases, one identical failure: they all regenerate the file from an in-memory model, and the damage lands wherever that model is coarser than the format.

Nobody can tell you what the AI actually changed. Word, Google Docs, Cursor, Grammarly, Lex — every one of them shows you AI edits *before* you accept and then loses the distinction forever *after* you accept [SS]. Cursor keeps attribution in an admin dashboard. Grammarly keeps it in a cloud report. Nothing is anchored to bytes, nothing travels with the file, and no reader can ever look at a paragraph and know who wrote it. In a world where an agent is going to touch your documents fifty times a day, that is a genuinely strange hole.

Everything you own is trapped in a view. Notion, Coda, Airtable — the view *is* the data. Export a Coda doc to CSV and you get, in their own users' words, raw disconnected strings with the formulas and buttons and canvas properties stripped off [SS]. This is the deepest problem and it is the one that suggests the answer, so I will come back to it in the next section.

And then the boring ones, which matter more than they sound: markdown tools are too complicated for normal people (Obsidian's own community says most of a million-plus downloaders never get past their first note [SS]), the pricing keeps betraying people (Notion cut free AI to twenty responses *for life* and raised Business ~20% [SS]; Microsoft's +43% Copilot bundling attracted a CMA probe [SS]), and importing from anywhere is a lie — the number one failure class in import tooling is not broken formatting, it is **import reports that claim**

success while losing files [fetched] . Obsidian has literally posted a \$500 bounty for a detailed import log and a \$5,000 bounty for Notion database conversion [fetched] , which tells you how unsolved this is.

That is the problem set. Nine things, all real, all evidenced. Now here is the one idea that addresses them together.

§2 The one idea

Here is the thing I would want on a wall.

The file is the only source of truth. Every app-like thing — the board, the calendar, the decision card, the dashboard, the published site, and every single AI edit — is a deterministic, reversible projection of that file, owning no state of its own.

I want to explain why this is a law and not a slogan, because when the research round went looking for the foundational concept it came back with something better than an opinion. It found that this exact line separates the tools that survived twenty years from the tools that became traps [\[→ c1-core-concept.md\]](#).

On the survivor side: org-mode's agenda, which has been running since 2003, is *computed on the fly from date tags in plain text files and stored nowhere* [\[SS\]](#). Obsidian shipped Bases as a native core plugin in 2025, and their own pitch is that the `.base` file only saves how you want to look at your notes — delete it tomorrow and you lose nothing but the view [\[SS\]](#). Ink & Switch's Potluck keeps, in their words, *"a clear separation between text and annotations... the original text freely editable"* [\[fetched\]](#).

On the trap side: Coda's export loses the formulas, Notion's formulas cannot even aggregate across rows, and a Jupyter notebook's meaning lives in invisible out-of-order kernel state — one large study found **under 4% of GitHub notebooks reproduce identical results** [\[SS\]](#). The view ate the data. There is no clean way out.

So the law is not new. What is new is that nobody has made it the explicit product primitive. Everyone rediscovered it as a side effect. And here is the part that made me actually happy about this: **the two hardest pieces of enforcing that law are already built in this repo**. The law is only credible if (a) writing back is provably reversible and (b) the view can always be stripped back to portable markdown. Those are exactly the byte-preserving splice writer and the cross-engine degradation certificate we shipped. So this is not a new bet. It is a name for the bet we already made.

The category sentence, if you want the one-liner:

Notion made the app the source of truth and trapped your data inside it. frontmatter makes the *file* the source of truth and lets every app be a disposable lens over it — provably, byte for byte, reversibly.

That inversion is a category. "We have a board view" is a feature. "No tool can ever hold your data hostage, because views are never canonical" is a category.

And it composes with the AI story rather than sitting next to it. If every view is a projection, then an AI edit is just another projection running backwards — it lands as a splice into the file, with its byte range recorded, reviewable and reversible. The AI does not get a special path. It gets the same path as a human dragging a kanban card, and the same guarantee.

53 What the market cannot do (and how long that lasts)

I want to be honest about the window here, because some of these gaps are permanent and some of them close in three months.

THE GAP	WHAT WE ACTUALLY VERIFIED	HOW LONG IT STAYS OPEN
Byte-exact structured editing on <i>both</i> halves of the file	All three teardowns regenerate; the two strongest each solved fidelity only on the half their editor doesn't model [measured]	The frontmatter-splice claim is durable (rewrite-level for them). The general claim erodes as they notice
Fidelity without a running daemon	OpenKnowledge's byte-contract only exists inside a live CRDT server — agent edits literally error out without it [measured]	Durable. It is architectural
O(edit) instead of O(document)	Their own source comment: full serialize+parse per edit, " <i>unbounded by doc size,</i> " fixing it " <i>needs a real incremental parser</i> " they don't have [fetched]	Durable
Render-fidelity certification	Nobody certifies cross-engine degradation. Email testing (Litmus at \$500/mo) proves the <i>testing</i> layer monetises while the <i>data</i> layer stays free [SS]	6–12 months. Publish the dataset before the design
The review loop on files you own	Google's public API cannot create suggestions at all [fetched] . Lex's track-changes is still "in development" [SS] . OpenKnowledge's comments are machine-local and never committed [fetched]	6–12 months
Byte-anchored, portable, reader-visible AI provenance	The absence check passed precisely: admin telemetry (Cursor), cloud report (Grammarly), repo sidecar (Agent Trace). The writing editors have nothing at all [SS]	Open
The rendered, evolving AI-output document	Memory tools store facts; consumer apps store sources; exporters store dumps. Nobody treats the <i>finished output</i> as a first-class living document [↪ aj4]	Open — this is the category to name
Session interchange between AI tools	No standard exists. OpenAI and Claude exports are mutually unreadable proprietary shapes, and every parser is reverse-engineered [fetched]	Open. Whoever publishes the open shape becomes the default
A file-native home screen	Obsidian's Homepage plugin has 1,294,057 downloads and its whole model is "home = a markdown note you own" — beating every bespoke dashboard plugin roughly 20× [fetched] . No product ships this natively	Open
Claim-level method + confidence	Across OKF, SKILL.md, llms.txt v2, MyST, .prompty — no spec claims it, and OKF explicitly argues <i>against</i> storing scores [fetched]	Open, and a renderer is the right shape to solve it

And the one that is already gone, so please do not position on it: **"an agent can read and write my markdown."** That is commodity. Bear, Craft, Notion, MDflow, GitBook all ship it; Obsidian's own agent-skills repo is at 47,418 stars and the skills-distribution CLI did 9.3 million downloads in a single week [\[fetched\]](#). Anything built on that sentence is dead on arrival.

The window read. The generic agent-markdown layer is commodity today. The "markdown editor with an agent harness" category gets crowded in roughly three to six months — OpenKnowledge alone is shipping about a hundred releases a week and went from 3,239 to 3,673 stars in 27 days [\[fetched\]](#). What stays open six to twelve months is fidelity, the review loop, and the Obsidian-power-user collaboration lane, because every

incumbent's incentives point somewhere else: Notion toward blocks, OpenAI toward artifacts-as-output, Google toward Docs, OpenKnowledge toward the team wiki.

There is a structural hedge worth naming. OpenAI *removed* Canvas in May 2026 [SS]. Their bet is that the document is agent output, not an editing surface. If "delegate, don't edit" wins, then the durable asset is not the editor chrome — it is the verification layer that checks what the agent produced. Which is MDMAX. So the roadmap deliberately keeps weight on the engine.

§4 The trick: markdown doing app work without app compute

This is the part you asked about specifically, and it turned out to be the most satisfying research of the whole round, because the answer is not speculative — it already ships in production at scale [\[→ c2-low-compute-app.md\]](#).

The mechanism everything rests on is the **fenced code block's info string**, and it is CommonMark by spec, not a hack. The spec says the first word of the info string is used to select particular treatment [\[fetched\]](#). Obsidian exposes exactly this as `registerMarkdownCodeBlockProcessor(language, handler)` — the info string names the renderer, the fenced body is the data slot [\[fetched\]](#). And crucially, an unknown info string just renders as a code block everywhere else. So it degrades by construction.

Think of it like a wall socket. The fence is the socket, the info string is the plug shape, and the body is the current. The document does not know what is plugged in, and if nothing is, you still see a perfectly readable block of text.

That framing gives you the security spine too. The AI emits *typed declarative data* into a slot; a fixed interpreter we shipped renders it. The AI never emits code. The moment you let the fence body be code — `dataviewjs`, `MDX`, `TiddlyWiki` macros — you have handed over the canvas and the "provably cannot corrupt" guarantee dies with it. So: hand the model a data slot, never a canvas.

Here is the computation budget, which I want written down so we never over-promise:

BEHAVIOUR	WHERE IT RUNS	MECHANISM
Text, lists, tables, callouts	Client, free	CommonMark + GFM
Checkbox input, toggling persists	Client + editor write	<code>- []</code> → source line rewrite. Dataview's TASK query is <i>"the only command in dataview that modifies your original files"</i> [fetched]
Inline edit of a typed field → YAML	Client + editor widget	Bases table cell writes back the frontmatter property [fetched]
State machine → kanban columns	Client	<code>status:</code> enum is the state; <code>groupBy(status)</code> is the columns; a drag is an enum write-back
Per-row computed column	Client	Bases formulas — <code>if()</code> , arithmetic, date maths [fetched]
Cross-note aggregates (sum, avg, median, stddev)	Client	Bases summaries / Dataview's <code>sum·reduce·average</code> [fetched]
Dashboard = query over a folder	Client, with a ceiling	In-browser index. Dataview advertises 100k notes but users report ~30-second queries past 3,000 [ISS]
Conditional render from a flag	Client	<code>if()</code> / <code>choice()</code> inline
Declarative diagram	Client + library	Mermaid, ~16 diagram types, text → SVG [fetched]
Declarative chart	Client + library	Vega-Lite JSON, or a <code>chart</code> fence into Chart.js [fetched]
Arbitrary interactive widget	Client + eval — REFUSE	dataviewjs, MDX. This is the lane we do not enter
Spatial canvas with x/y	Hard boundary	Position data is not linear text. Sidecar JSON, or don't
Real relational queries, joins, transactions	Server	An in-browser index cannot
Multi-user concurrent state, presence	Server	A file-per-note model is single-writer by nature
Classify, summarise, generate	LLM	Not a render-time operation, ever

Two things I want to pull out of that table.

First, the founder framing of "client / server / LLM" is missing a fourth lane, and it is the important one: **client-side but arbitrary code**. Our whole defensibility lives in staying in the fixed-interpreter lanes and refusing the eval lane. "Doesn't require much computation" really means "doesn't require *arbitrary* computation." A closed vocabulary of render verbs, not a sandbox.

Second — and this is the practical instruction — we should **buy the semantics of Bases and Dataview, not their code**. Those two engines already define the computation surface users expect from a markdown app. Dataview is MIT [\[fetched\]](#), so we can actually take the query parser rather than just admire it. Match that surface as a stable spec, and publish the ceiling honestly so the render promise never quietly reaches into server or LLM territory.

§5 The product, layer by layer

I am going to describe this as five layers, because they stack — L0 has to be solid before L1 leans on it.

L0 — The simplest editor anyone can trust

Most of this is already shipped from the `sgnk-md` base: four view modes, file tree, tabs, formatting bar, slash commands, vim keys, wikilinks, KaTeX, Mermaid, exports, PWA and Tauri builds. What is missing is the trust surface, and that is Phase 0.

- **Sync you can watch.** A visible sync chip, a conflict inbox showing mine/theirs/keep-both, version history with *named* versions, and a "changed since you last opened" banner. That last one is nearly free for us — it is a diff between two shas, where Google needed bespoke infrastructure [SS]. The exit test is a demo: two devices editing offline, converging with zero loss, and the user *watched it happen*.
- **Local history.** Every save is a revision, default on, with a ten-second merge window and a merged timeline of git commits plus local saves plus AI edits, filterable by who made it. Restore a *section*, not just a whole file. And no silent expiry — JetBrains wipes local history after about five days and that is precisely the anti-pattern, because the one feature whose entire job is trust cannot have a quiet trapdoor [SS].
- **The WYSIWYG rules, taken straight from Typora's own issue tracker** [fetched]. Markup reveals on *keystroke*, never on click (#443). The caret maps to rendered geometry, and backspace eats text characters, never delimiters (#2271). One explicit reveal policy toggle, not per-element emergent behaviour (#1317). And caret plus scroll survive every mode switch — which is nearly free for us because the `OffsetMap` already maps positions across representations. That is a real differentiator we get cheaply.
- **Doc Health**, copying VS Code's four-surface Problems model: a status-bar count, a filterable panel, inline marks, and F8 cycling with quick fixes. Vault-scoped checks on by default, publish-dependent checks off. What we do *not* copy is Grammarly's opaque document score — we do binary named gates, never "your writing is 82/100."
- **Importers with a verification report**, which is the actual product here. The seven panels: a reconciliation table that sums to a census counted from the *output* filesystem (not the conversion loop — that is how the lying reports happen), a per-item failure ledger, a rename ledger, a link-resolution audit, an attachment table with HTTP statuses, construct-downgrade declarations, and a metadata-preservation matrix [fetched]. Count parity alone is provably insufficient, because content loss hides inside conversions that report success.
- **Home is HOME.md** — a real markdown file rendered with custom-render blocks. More on this in the screens section.

L1 — Custom renders: one file, many surfaces

The mechanism is designed already (the dispatch table with `render/degrade/refuse`). What activates it is a frontmatter key, which is a nice bit of symmetry given the product's name.

The profiles, in build order: **kanban** (columns are the values of a frontmatter field; drag rewrites that field), **decision** (the ADR profile — and MADR already stores status, date and decision-makers in YAML frontmatter, so we render it rather than inventing anything [fetched]), **calendar** (drag a card to another day, it rewrites `date:`), **corkboard and outliner** (Scrivener's planning views, as renders over `synopsis:` and `status:` keys), **slides** (Marp-compatible), **declarative figures**, **brand book**, **project map**, **live site**, and **flow** last, with derived layout only.

On kanban specifically, the opening is real: the flagship Obsidian Kanban plugin has 2.6 million downloads, its last release was 26.9 months ago, and the repository now lives in an org literally called *community-archive* [fetched]. The file convention it established — ## Lane headings with - [] cards — is a de-facto standard that at least three plugins read and write. We adopt the convention and win on the write path, because a drag in our editor is one splice move on a syntax-tree span, byte-identical, which no ProseMirror-based board can promise.

There is a prerequisite that is embarrassingly small and blocks the whole track: components.tsx:133 uses /language-(\w+)/ to pull the fence language, and \w excludes hyphens, so every hyphenated language collides with its prefix. One character class. That is the first commit of this track.

L2 — The AI protocol

This is where the product stops being an editor.

The MCP server, tools-first. The July 2026 MCP revision went stateless and deprecated Roots, Sampling and Logging, which tells you tools are the lowest common denominator [fetched]. So: a small set of consolidated verbs — land, search-vault, read-slice, splice-edit, cert-check — hard-capped well under twenty, verdict-first responses, and refusals that name the rule they violated with a corrected example. The plumbing is table stakes now. The pitch is what rides on it: **every agent edit goes through the splice writer, so the agent cannot corrupt what it edits.**

The capture verb. One call, land(), that any MCP-capable agent can make:

```
land({ path?, type, title, body_md,
      source: { tool, model, conversation_url, session_id },
      base_version?, mode: create | patch | rewrite })
→ { LANDED | VERSIONED | REFUSED_CONFLICT | NEEDS_TARGET,
    path, version, url, bytes_written, cert }
```

The design rules behind that shape came from studying every chat-to-artifact system that exists [↪ aj2]. Identity is the *path*, stated explicitly in every call — because on claude.ai, where identity is inferred from phrasing, "it made a new artifact instead of updating mine" is the single most documented failure. Two edit sizes are protocol modes, not prompt etiquette. And base_version enforces read-before-patch, which structurally kills the drift bug where the user hand-edits a document and the model keeps talking about the version it remembers. Bolt's own system prompt says it plainly: always edit the latest content [fetched]. The file is the memory; the model's memory of the file is a cache to invalidate.

Provenance. On every accept, the splice records the byte range plus {contributor, model, promptDigest, sessionRef}, and a "Show AI ink" toggle tints AI-written spans for any reader. Granola already proves people like the legibility — they render AI text grey and your text black [SS] — but theirs vanishes on export. Ours is byte-anchored, travels with the file, and interops with Cursor's Agent Trace format. I want to be careful with the claim wording though, and the research checked it for me: this is **not** "first AI attribution ever" (Cursor and Grammarly exist). It is *the first markdown editor with byte-anchored, document-portable, reader-visible provenance*. That is the version that survives scrutiny.

Implicit telemetry. Whether the user kept, edited or reverted an AI edit — measured from the document, with no rating buttons. This shape is mandatory, not preferred, because our own AIOS proved that explicit feedback channels starve: the accepted field was filled on 7 of 688 recent rows even with one motivated expert using it daily [measured]. Meanwhile the machine channels filled themselves — 23,778 routing decisions logged in the

same period. Build every feedback-dependent feature on what the system can observe, never on what a user is asked to declare.

The AI surface ranking, straight out of the complaint corpus: deliberate inline edit beats review-moded agent beats on-demand chat beats ghost text — and ambient AI buttons are net-negative, to the point where Notion users write ad-blocker rules to kill them [SS]. So ghost text defaults to Zed's *subtle* mode (visible only while a modifier is held), there is a loud global AI kill switch, per-folder exclusions, a snooze, and propose-first as a named default mode.

Also in L2: citation-gated vault answers that refuse when they cannot ground themselves (Advox's fail-closed verifier, which we already built), document session continuity, the routing gate that keeps ~90% of AI operations on the cheapest capable model (that is margin, measured across 23,778 live decisions [measured]), the certificate distributed as npx, MCP, a generated skill and a GitHub Action, and mdmax explain --as <consumer> — the differ that shows you exactly what your file does to a model's context window, which nobody else has anything like.

ACP comes later and deliberately: forty agents are registered against it including an Anthropic-co-authored Claude adapter, and session/load|resume|list is the only shipped multi-vendor session-continuity semantics anywhere [fetched]. But ACP agents are local subprocesses over stdio, so it needs a desktop surface first. MCP now, ACP when we have a local build.

L3 — The markdown-OS features

These are the conventions our own AIOS already runs on, turned into user-facing features [measured → i5]. Schema profiles (user-definable frontmatter schemas the editor validates and the AI reads — knowledge/meta/schema.md is the working prototype). Append-only blocks. Auto-generated index pages. Machine-write zones, fenced regions where the AI may rewrite but never outside. A token-budget meter showing what an agent-facing file costs in context. **Provenance chips and evidence-tier fields** — the {value, source, tier, re_verify_cmd} discipline our campaign system already gates publications with [measured], and which no competing editor has in any form. Doc staleness detection. Document CI. And eventually, user-authorable automations as documents, because the automation format is literally the product's format.

L4 — Publishing and the GTM loop

Post-as-document: one .md compiling to a LinkedIn PDF, an Instagram carousel, a blog article, a thread and a status card — the campaign pipeline already proved this works, ten complete episodes built and gated in about a week [measured]. Scheduled publish as draft-and-queue with a human confirm, never unattended. The AEO linter, honestly framed as content-quality coaching (the Princeton work measured 25–40% visibility lift from quotes, statistics and citations [SS] — while the "serve markdown and get cited" claim is measurably refuted, so it will never appear in our marketing). And document quality gates, never scores.

§6 Every screen

Twelve screens. For each one: what it is for, what is above the fold, the main interaction, the empty state, and where AI lives [→ c6-screens.md] .

1. First run. One choice — create a vault, open a folder, or try a sample — with no email field and no OAuth wall. Obsidian's own help text is the north star here: *"No account required... Privacy is the default, not a setting"* [SS] . Landing goes straight into a seeded `welcome.md` that is itself a live-rendered file, so editing it is the tutorial. Auth defers to the first feature that needs it. What to avoid: the multi-slide personalisation questionnaire, which for a document tool just delays the one thing that proves the product — a blinking cursor in a real file.

2. Home (HOME.md). Two jobs only: resume the last thing, start a new thing. Band one is search plus a blank-first create row (Blank tile first and ghost-styled, then templates, then a gallery overflow — the Google Docs convention exactly). Band two is a recents grid with real previews, with "landed today" and a needs-review lane at its head. Sections are hideable and reorderable from v1, because retrofitting section plumbing later is expensive. The empty state is the templates row promoted — never an illustration. And the hard rule: home must never steal the startup slot from "my last file, my cursor position." VS Code's forced welcome tab generated years of make-it-stop issues [SS] .

3. The editor, four modes. Live (default, WYSIWYM, markup reveals only when the caret enters a span), Edit (raw source), Split (scroll-synced), Read. `Cmd+E` cycles. Chrome fades on typing and returns on mouse-to-edge, which is how a busy three-pane frame stays premium. No persistent formatting ribbon — a selection bubble and a slash menu. The right rail order: **Properties first** (the typed frontmatter editor, since that is our name doing real work), then Outline, Comments, History, Tags, and AI-Edit.

One reconciliation with your mockup that I want to flag rather than quietly change: the mockup has a docked AI writing box at the bottom of the rail. Every shipped convention puts the AI prompt *at the cursor*, and a rail-docked prompt reads as bolted on. My recommendation is to keep the AI-Edit *tab* — as the place where pending AI hunks are reviewed, which is correct — but move the prompt to the selection. Your call, but that is what the evidence says.

4. The render views. Kanban (columns are field values, drag rewrites the field, and card order does *not* go into hidden block IDs in the file — spatial state belongs in a sidecar). Decision (status chip driven by frontmatter, body rendering Context → Drivers → Options → Outcome → Consequences, straight off the MADR template [fetched]). Calendar (drag to reschedule writes `date:` , edge-drag writes a range). Site (left nav, auto-TOC from headings, hover previews, and search indexing **off** by default).

5. The review surface. One grammar for everything: human suggestions, AI edits and sync conflicts all arrive here as hunks. A mode dial — Edit / Suggest / View — because suggesting is a mode, not a special AI thing. Gutter bars mark changed regions; a suggestions list carries operation sentence, author (human or named agent), timestamp and a reply thread. The adjudication ladder goes per-hunk, per-suggestion, then all-shown-under-filter, with accept-and-advance traversal and a preview before any bulk action. Accept is a splice with a `Co-authored-by` trailer. Resolution is an authored thread event, never a silent boolean. And when an anchor's text has moved, we badge the orphan and preserve the quote — Word silently deletes those comments, which is exactly the wrong answer.

6. AI panel and inline diff. Nothing persistent; the AI surface is summoned. Prompt appears at the selection. Insertions show as ghost text, edits as tinted inline diff with per-hunk accept — that per-hunk control is the

single most-demanded feature in AI editors, and both Cursor and Windsurf got burned when they regressed it [fetched]. Three verbs always in the same order: Accept, Discard, Try again. No auto-apply, ever.

7. Publish and share. A two-tab popover — Invite and Publish. One toggle produces a URL with a copy button; options live behind a disclosure; indexing off by default. Post-publish is a first-class state with URL, Copy, View and Unpublish in the same place. And unpublish must revoke *immediately* — we should only promise what the revocation machinery actually enforces, which after commit d50a6b2 it now does.

8. History. Two timelines in one surface: named versions (git-grade, shared) and local history (per-save, private). Preview before restore, always. Section-level restore, not just whole-file. AI edits are first-class actor-labelled entries, so you can filter to "AI only" and audit every agent change.

9. Doc Health. The four VS Code surfaces, with a *green* all-clear zero state rather than a blank panel. Unreviewed AI edits are themselves a diagnostic category.

10. Settings. Searchable, sectioned, and stored as versioned files in the vault where possible. The AI section is BYO-key with a provider dropdown, a Validate button and a model picker. Keys never go in committed files. The rules files open as plain markdown — Cursor gave .mdc a special editor UI and users actively look up how to turn it off, which is dialect betrayal happening inside an AI IDE's own config format [SS].

11. Pricing modal. Contextual, naming the blocked action, with the unlocking plan pre-highlighted. Two plans maximum in the modal; the full comparison lives on the pricing page. When the gate is an AI limit, show the meter and the honest option: add a BYO key and keep going free, or upgrade for hosted convenience.

12. Team dashboard. Teamspace sidebar with shared and private sections, a review queue across the team's files, and a deliberately lean role model — Admin, Member, Guest, no custom roles (Linear's restraint is worth copying). The agent is a standing reviewer, GitBook-style. And no document freezes for governance — use branch-protection semantics instead.

The grammar that ties it together: the file is the unit, the render is a lens, and every mutation — human, AI or sync — flows through one review surface. Cmd+K is the universal palette with #, @ and : goto operators. Cmd+E cycles modes. Space summons AI at the cursor. Tab accepts, Esc rejects. And every empty state is the next action promoted, never an illustration, because the blank page is the documented abandonment killer.

§7 Who pays

Two motions, one substrate. This is the thing that resolves the B2B-versus-D2C question instead of forcing a choice.

Acquisition is D2C-shaped. What people fall in love with is the individual magic — my one file just became a board, and my AI edits are visible and reversible. That love plus content is a solo founder's only affordable customer acquisition. You cannot run an outbound enterprise motion alone.

Monetisation is B2B-shaped. The same file the individual loves is the file a team pays for. Stripe's data on solo founders is blunt: by month 24, the median solo B2B founder's revenue is more than **4× the median solo B2C founder's** [SS]. So lean B2B for revenue — but through *self-serve* teams of two to twenty who pay by card, not through procurement.

The four B2B profiles, in the order I would chase them [→ c3-b2b-wedge.md] :

1. **Dev-tool and API startups (2–20 people).** The highest-fit beachhead, because they already live in this substrate — `gray-matter`, the canonical frontmatter parser, does **35.8 million npm downloads a month**; `js-yaml` does **1.23 billion** [fetched]. Their hook documents are the docs site, the changelog, the status page, and the runbooks and ADRs that have *no good tool today at all*. The pitch is consolidation: a representative stack today is Mintlify \$250 + GitBook \$65 + Statuspage \$399 + LaunchNotes \$249 = **\$963 a month** across four vendors that do not share a source of truth and drift apart [SS]. Anchor at \$20–40 per editor seat, or \$150–300 per team.
2. **Agencies and studios (2–15 people).** One file per client that renders as a client-facing status dashboard, updated by the agent from work logs, with typed frontmatter rolling up into an agency-wide view. This is literally Zephyrus's own shape, which means we can dogfood it honestly. Price-sensitive market: flat \$29–49/month or \$9–19 a seat.
3. **Support-heavy SMBs.** The cleanest ROI story — knowledge-base deflection runs 18% median (up to 40–60% with AI), and a SaaS support ticket costs \$25–35 [SS]. \$99–249 per knowledge base. But fierce incumbents, so land it *after* the substrate story is proven.
4. **SMB internal ops.** Wiki, SOPs, runbooks. \$8–15 an internal seat.

Two framing corrections I want to hold onto. First: **do not lead with deflection or "structured data."** Intercom, Zendesk, Document360 and GitBook all sell exactly that pitch. It is our *proof*, deployed after a team is inside — not our opening line. Our wedge is one non-corrupting file that is editor, AI workspace, rendered surface and audit trail at once.

Second: the audit trail is the part incumbents genuinely cannot copy quickly. Because the splice writer attributes changes at the byte level, **the file is its own audit log** — who changed what, human or AI, and when. One vendor's case study had an audit log shorten a sales cycle from four months to six weeks; Vanta and Drata charge \$7–30k a year for continuous audit trails [SS]. We get a slice of that as a byproduct of *how we write files*, not as a bolted-on compliance module. And "the AI edited your ops file, here is cross-engine proof it corrupted zero bytes" is a governance claim no Notion or Confluence AI can make.

What to avoid until there is a team or funding: SSO, SCIM, SOC2 and the enterprise knowledge-base market. Those are four-month sales cycles against funded incumbents, and one person cannot service them.

§8 Pricing

Your draft is Free / ₹299 / ₹699. The research pressure-tested it and came back with three changes and one thing to keep exactly [\[→ c4-pricing-inr.md\]](#).

First, a correction that matters. The brief assumed roughly ₹83 to the dollar. The actual rate is about **₹95.4** [\[SS, August 2026 – re-check live before publishing anything\]](#). So ₹299 is **\$3.13**, not \$3.60, and ₹699 is **\$7.33**, not \$8.40. The rupee moved ~13% and it changes the conclusion on the top tier.

₹299 is right. Keep it. The correct anchor is not "PPP percentage off a dollar price" — it is what Indians already pay for AI productivity, and that band is now sharply observable: ChatGPT Go launched at ₹399, Gemini AI Plus runs ₹199 intro to ₹399, Netflix India sits at ₹149/199/499/649, and the Indian micro-SaaS starter band is ₹299–499 [\[SS\]](#). ₹299 sits below the ChatGPT Go anchor, at the floor of the starter band, on a strong psychological point, and **undercuts Notion India's ~₹670 by about 55%**. Worth A/B testing ₹249, which lands more decisively inside the PPP band, since India is a distribution play anyway.

₹699 is the weak number — move it to ₹599. At \$7.33 it is above every India AI anchor, above Netflix Premium at ₹649, and only 19–27% below a \$9–10 global tier, which makes it near-parity rather than an India price. ₹599 sits under the Netflix ceiling, is a clean 2× step from ₹299, and is ~30% off the world tier. The alternative — and honestly I like this better — is to make the top SKU a **flat commercial-use licence at about ₹3,999/year**, mirroring Obsidian's model, rather than a monthly power tier at all.

Lead with annual billing in India. This is not an upsell tactic, it is fee survival. A merchant-of-record like Dodo charges 4% + \$0.40, and that flat \$0.40 alone is **12.7% of a ₹299 monthly charge** — versus about 1.5% on a ₹2,499 annual one [\[SS\]](#). So: ₹2,499/year for Pro (~₹208/month), ₹4,999/year for the top tier.

Does ₹299 survive AI costs? Only by architecture, and the architecture we already chose is the one that works. Net revenue per ₹299 subscriber is about ₹246–249 (~\$2.60) after 18% GST and processing. A healthy margin leaves roughly \$0.40–0.60/month for inference. That buys about **450 assists on cheap models** (Gemini Flash, DeepSeek-class), about **77 on Haiku**, and about **5 frontier assists** [\[SS pricing, arithmetic ours\]](#). Which means: bundled unlimited AI at ₹299 is impossible, and metered-plus-BYO-key at ₹299 is comfortable. Free tier carries **zero hosted credits** and unmetered BYO-key, so our heaviest free users cost us nothing. Pro carries a small hosted allowance defaulted to a cheap model with a visible meter; frontier work is BYO-key or metered pass-through.

And a strategic note: **we cannot win India's AI price war and should not try.** ChatGPT Go went *free* for Indian signups, Gemini is subsidised at ₹199, Perplexity Pro is free via Airtel [\[SS\]](#). A bootstrapped studio does not out-subsidise that. Which is itself the argument for BYO-key, and for India being top-of-funnel rather than the revenue engine.

The tables.

India:

TIER	MONTHLY	ANNUAL (PUSH THIS)	CONTENTS
Free	₹0	₹0	Full editor, unlimited docs on your own repo, offline, splice guarantee, BYO-key AI unmetered , fair-use publish, on-device checks, zero hosted credits
Pro	₹299 (A/B ₹249)	₹2,499	Publish extras, live editing, hosted convenience, small metered AI on a cheap model with a visible meter, AEO linter
Power	₹599 (not ₹699)	₹4,999	Multi-site publish, higher meter including metered frontier, priority
Work	—	₹3,999/yr flat	Commercial-use licence — support and compliance, not more features

World:

TIER	MONTHLY	ANNUAL	ANCHOR
Free	\$0	\$0	Obsidian free-forever
Pro	\$5	\$48	Obsidian Sync \$4/\$5, HackMD \$5/seat, Bear \$3, Ulysses \$6 — all [SS]
Power	\$10	\$90	Obsidian Publish \$8/\$10, Craft \$8
Work	—	\$50/yr flat	Obsidian Commercial parity

₹299 against \$5 and ₹599 against \$10 are both about 37% off, which is a coherent ladder that lands on Indian psychological price points without looking like mechanical PPP.

Which market first? Distribution India-first, revenue global-first. India has the largest and fastest-growing developer base in the world (21.9 million GitHub contributors, +5.2 million in one year [SS]), a real paid-app inflection (+35% YoY consumer app spend, productivity and AI leading [SS]), and we are based here with UPI and Razorpay native. Feed all of that into the free tier. But there is still **zero category-specific willingness-to-pay evidence for markdown tools in India**, the AI price here is collapsing to zero, and low-ticket monthly INR economics are punishing. Notion refuses to price in rupees and still grows in India — proof that India can be an audience without being the revenue centre.

On rails: Razorpay for India (cheap, UPI-native, but we own GST filing) plus a merchant-of-record for the world; or a single UPI-capable MoR like Dodo doing both. What matters most is not the discount — **it is that UPI exists at checkout**, because international gateways without it lose 30–40% of Indian checkouts [SS]. Paddle and LemonSqueezy do not support UPI at all.

59 Marketing

Two funnels. They need different content and probably different people watching them.

Funnel one — trust, for developers and prosumers. The line is *"the markdown source of truth AI can't corrupt."* These people live on Hacker News, r/ObsidianMD (about 344,000 members), Discord (about 195,000), X and YouTube. The content is evidence-first: the 907/907 corpus result, the foreign-vault fidelity run, a ten-second demo diff against a competitor's shipped product. Our existing campaign practice already manufactures exactly this kind of post — the atlas even contains frontmatter-native material already [measured].

Two things happened in the last six weeks that make this funnel warmer than it was. The review-loop wedge got independently validated twice: Markleft ("how I review Claude's markdown plans", Show HN) and OzBrain (92 points on HN), whose founder pitches *literally* "the diffing, versioning and audit log of what was changed, by what agent and why" as the paid product [fetched]. And "Serve Markdown to AI Agents with Accept Headers" hit **175 points with 108 comments on 2026-08-26** [fetched]. The conversation is happening right now.

Funnel two — beautiful documents, for writers and students. *"Beautiful documents from plain text."* This is the Instagram-and-TikTok funnel, and custom renders demo in fifteen seconds of vertical video. Notion's Gen-Z growth ran through exactly this kind of aesthetic template content [SS]. But I want to be honest about two caveats: no dev tool has ever grown Instagram-first, and our current content pipeline mines the *system record*, so it would never surface aesthetic material on its own. This funnel needs a new content vein and a smaller budget until it earns more.

Channel discipline. The Obsidian community's code of conduct means the only admissible framing in the biggest watering hole is "integrates with your vault," never "Obsidian competitor" — break that and the launch gets removed rather than debated. The companion "Open in frontmatter" plugin is the distribution play; Relay proved the path with 172,544 downloads of a commercial service's bridge plugin [SS].

Cadence, grounded in what we actually produced. Three LinkedIn posts a week plus a weekly canonical blog post, and two Instagram carousels. The campaign system built ten complete multi-surface episodes in about a week, so supply is not the constraint [measured]. What is *not* proven is sustained cadence — exactly two posts have ever shipped [measured]. So the survival mechanisms matter more than the plan: never miss twice, keep queue depth at two or more, and read no metrics before post twenty.

And the move I like most: **run the launch calendar inside frontmatter itself.** Calendar profile, evidence-tier fields on every post, draft-and-queue publishing. The GTM engine becomes a continuous product demo, and the content pipeline's own biggest gap — an ongoing capture habit — becomes a product feature.

One thing we should never say: "markdown editor." Every Show HN with that name turns into a thread of free alternatives. The category winners all renamed the category first — Obsidian sold *ownership*, Notion sold a *workspace*, Linear sold *speed*.

§10 The build order

Six tracks. R0 comes before we market any fidelity number, because right now we would be over-claiming.

R0 — Engine truth. The foreign-vault run is the most useful thing we did all week and it found real work `[measured → h2]`. Across **7,959 frontmatter files from five different authors' public vaults**: zero corruption, zero throws — the guarantee holds on other people's data. But **83% of those files get refused** for publishing, and 6,613 of 6,614 refusals have a single cause: a block sequence written at zero indentation (`tags:` then `- item` starting at column zero). That is spec-valid YAML, it is PyYAML's default output, and it is idiomatic in the Chinese vaults we sampled. Recognising a dash-item as a continuation of the preceding key recovers **99.98%** of them. Then: a bare-CR fence (`---\r`) misses our open-fence regex, so a lone `set` prepends a second frontmatter block — same family as the BOM bug we already fixed, and invisible to the current oracle because set-then-delete cancels it out. And `SAFE_KEY` is too narrow for real vaults, where `date created` appears in 812 of 957 files in one garden. None of these are corruption. All of them are availability, and the fix list is short. Also in R0: the six construct-detector defects (they gate the AEO linter *and* kill-condition 4), CI (we have no `.github/` at all, which is awkward for a product that wants to sell document CI), and the CJK problems — `countWords` undercounts Chinese by 1.7–2×, and MiniSearch's CJK recall collapses to 18% mid-clause `[measured]`.

T0 — Trust surface. Sync chip, conflict inbox, named-version history, since-you-last-opened banner, background auto-sync, local history. Exit: the two-device demo, watched.

T1 — Tenancy and launch. Identity and multi-tenancy (HQ's pooled-RLS spine and CareerOS's entitlements engine are the lift — this "gap" has shipped code in two sibling repos), GitHub App auth, multi-vault, mobile pass, HOME.md, quick capture, and the Obsidian companion plugin.

T2 — Renders. The regex fix, delete the dead table code, kanban read-only, kanban bidirectional gated by a zero-dirty oracle, decision, calendar, corkboard, compile profiles, publish-with-profiles, `mdmap check` CLI.

T3 — AI protocol. MCP server and `land()`, the review loop (where AI edits arrive as suggestions in the human grammar), provenance and implicit telemetry, the differ, cert distribution, citation-gated answers, session continuity, ACP client once there is a desktop build, and the open session-interchange format published.

T4 — Capture funnel. The chat-side skill and paste-inbox, ChatGPT and Claude ZIP importers (where the verification report *is* the demo), the promotion loop, retro-capture of whole conversations into several typed documents.

T5 — Content and GTM. Land Markex's uncommitted fix tree, run the LinkedIn Documents-API probe, post-as-document, then the launch calendar running inside the product.

T6 — Scale. Offline-first, WYSIWYG, share roles, comments, suggest mode, teams.

The sequencing law: the differentiation tracks — R0, T2, T3 — have to land inside the six-to-twelve-month window before the category crowds.

§11 What I will not build

Writing this down because the expensive mistakes are all additions.

No new format, sigil or dialect. No plugin marketplace — VS Code's own wiki calls extensions the number one performance suspect, and Typora's most-requested feature (251 votes for plugins) sits against a product people love *because* it has none. No tree-of-record, ever, even for one feature — `blocksToMarkdownLossy()` is a real function name in BlockNote's API and it is the market confessing. No block IDs written into files, no columns, no synced-block bytes. No Likert writing scores. No ambient AI buttons. No un-disableable AI. No silent auto-landing into a curated vault. No streaks or badges. No format-on-save default. No opaque credit repricing — Cursor apologised for one, Windsurf churned twice, Notion bundled and got roasted. No bespoke canvas as the centre of gravity. No claims about a learning loop until a decision demonstrably bends on real data. And none of the AIOS internal machinery as consumer UI: the learned-rules ledger, the debate panels, the shadow-promote ladder are excellent for running *us* and would be a trust burden shipped to anyone else.

Also on the list, from the earlier rounds and still true: the mother-markdown container (45 transclusions in 25 MB of my own vault, all of them documentation of the feature, none of them use of it), graph-view investment (zero of 89 feature requests mentioned it), and novelty-for-its-own-sake on the top tier.

§12 What could kill this

- If the kanban write-back cannot pass a no-op-dirties-zero-files oracle on real boards, it ships read-only. No exceptions.
 - If the R0 sequence fix cannot get the foreign-corpus refusal rate down, publish stays refusal-honest and we say so publicly rather than guessing at YAML.
 - If `mdmap check` runs on this repo and on the md vault and makes nobody want to fix anything, that thread stops.
 - If the AEO linter cannot be correlated with any measurable signal on our own published corpus, it is a writing aid — rename it or drop it.
 - If funnel two produces nothing after ten posts, fold back into funnel one.
 - The review surface is a trust contract, not a feature — any change to it is treated as a breaking API change. Both Cursor and Windsurf regressed it and both got publicly burned.
 - Any "first" or "only" claim gets one more verification round before it ships. We already caught one of mine that way.
-

§13 What to read before writing code

The full license-flagged bibliography is in `c5-build-refs.md`. The eight to read first:

1. `@lezer/markdown` (MIT) — the incremental parse tree. `SyntaxNode` byte offsets are our span-addressing primitive. Most load-bearing read in the list.
2. **The OKF spec** (Apache-2.0) — Google formalising our exact bet: a directory of markdown files with YAML frontmatter, path as identity, `type` as the one required field. Align or consciously diverge.
3. `@codemirror/merge` (MIT) — the review loop as a shipped component. Highest reuse per hour on the list.
4. **Hypothesis's** `match-quote.ts` + `approx-string-match` (BSD-2 and MIT) — the anchoring mechanism that lets an AI target survive human edits.
5. `mdast-util-to-markdown` (MIT) — read the *enemy*. This is the serializer that normalises, and its exact loss points are what our splice writer's value is measured against.
6. `@sanity/diff-match-patch` — note this carefully: Google's original diff-match-patch is Apache-2.0 and safe, but the repo has been archived and unpublished since 2020. Use the maintained Sanity fork.
7. `@modelcontextprotocol/sdk` — currently mid-relicense from MIT to Apache-2.0.
8. `obsidian-dataview` (MIT, 9,300 stars) — the closest prior art to our whole rendering bet, and MIT means we can actually take the query parser rather than just admire it.

Two license landmines: **obsidian-kanban is GPL-3.0 and abandoned** — read the board file-format convention, copy zero code. **anthropics/skills has no license file at all**, which means all rights reserved — mirror the SKILL.md convention, copy nothing.

And one operational surprise worth knowing: **the entire public CodeMirror GitHub org was archived on 2026-04-15** — 55 of 57 repos `[fetched]`. This is not death; npm is very much alive (`@codemirror/view` v6.43.9 published 2026-08-16). But it means the source is read-only, you cannot file issues upstream, and we should budget for vendoring risk on any CM6 bug we hit.

§14 Decisions that are yours

1. **sgnk-md versus frontmatter.** frontmatter is a clone of md; md is still live at md.sgnk.ai as your daily vault; every editor fix now has to land twice or silently fork. My recommendation: frontmatter becomes the sole product codebase, md stays as vault data and first customer, and md.sgnk.ai eventually redeploys as a frontmatter instance. Also, ecosystem.md has no frontmatter card at all — the source of truth does not know our peak product exists.
 2. **The CRDT contradiction.** §4b of the product plan says no peer CRDT; Pillars 3 and 5 still list Yjs multiplayer. §4b is later and explicitly a resolution, so it should govern — but the feature tables were never updated. Needs a ruling.
 3. **The name.** Risk is medium-high, and high if we ship as bare "Frontmatter." The senior user is same-market, same-channel, same-audience (a git-blog CMS aimed at exactly our segment #2), active since 2019, with 80,527 installs [fetched]. Two openings: its author's 2026 README openly invites company sponsorship, and **MDMAX is collision-clean** (npm unregistered, GitHub near-empty [fetched]). Options are keep-with-qualifier, flip the hierarchy so a coined mark is the brand and "frontmatter" stays the feature word, or keep it with written rename tripwires. The evidence argues against the bare unqualified name. This one wants counsel before we spend on brand.
 4. Max-tier contents, the export contract, and the OAuth scope (read:user grants no repo access, which blocks the GitHub-App path as written).
 5. The two PATs from an earlier session are still unrotated.
 6. Two thirty-second verifications only you can do — the AGENTS.md Linux Foundation announcement and the Otterly zero-citations experiment both sit behind bot walls that refused every automated path I have.
-

§15 What is still unverified

I would rather write this list than have someone quote a soft number back at me later.

Everything tagged [SS] is a search summary and nobody opened the page. The Mobbin visual design pass is still outstanding — the taint gate blocked it in both sessions, and it is session-scoped, so a fresh session recovers it. Reddit beyond r/ObsidianMD's top-of-month is rate-limited (the RSS path works, so it is a patience problem, not a wall). Mintlify's current Pro figure renders in JavaScript so curl cannot see it. And the OpenKnowledge WYSIWYG lane is a source verdict, not a live-browser test — their agent write path I *did* execute and it is genuinely byte-perfect, which is worth saying out loud rather than pretending they are careless. They are not. They just chose an architecture that caps them where we are not capped.

So that is the whole thing. To be honest the part I keep coming back to is not any single feature — it is that the law in section 2 was already half-built before we went looking for it. We shipped a byte-preserving writer because mangled YAML annoyed me, and a degradation certificate because I wanted to know what breaks where. It turns out those two are exactly the enforcement mechanism for "the file is the truth and every view is disposable," which is the same line org-mode found in 2003 and Obsidian rediscovered in 2025. Pretty good place to be starting from.

The next move is small and unglamorous: fix the zero-indent sequence case so the writer stops refusing four out of five real files, and get five people to actually try the thing. Everything else in this document is downstream of those two.

— Sagnik :)